# Real-world (build and compare the supervisor version) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

**Story — `architecture_comparison_practice.py`:** this builds the supervisor version of the exact same task, measured the exact same way as the sequential version, so the "why a supervisor instead of a fixed sequence?" question gets a real, measured answer instead of a feeling. The sequential stages ( `research` , `write` ) and a `run_sequential()` are included inline here too, so this one file runs its own comparison standalone. **If not:** Project 4's "Plan Before You Code" step would be defending its design choice with guesses instead of numbers.

## Solution

`run_supervisor()` mirrors `run_sequential()` exactly — same fields, collected the same way — so the comparison is fair.

```
# architecture_comparison_practice.py
import time
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END

model = ChatOpenAI(model="gpt-4o-mini")


def research(topic: str) -> dict:
    start = time.perf_counter()
    prompt = f"Research the top 3 causes of {topic}. List them briefly."
    response = model.invoke(prompt)
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": time.perf_counter() - start,
    }


def write(research_text: str) -> dict:
```

```
    start = time.perf_counter()
    prompt = f"Write a 3-sentence summary of this:\n\n{research_text}"
    response = model.invoke(prompt)
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": time.perf_counter() - start,
    }


def run_sequential(topic: str) -> dict:
    research_result = research(topic)
    write_result = write(research_result["text"])
    return {
        "routing_log": ["research", "write"],
        "total_tokens": research_result["tokens"] + write_result["tokens"],
        "total_seconds": (
            research_result["seconds"] + write_result["seconds"]
        ),
    }


# ---- the supervisor version: the same two stages, driven by a graph ----
class TeamState(TypedDict):
    task: str
    research_text: str
    summary: str
    routing_log: list
    total_tokens: int
    total_seconds: float


def research_node(state: TeamState) -> dict:
    result = research(state["task"])
    return {
        "research_text": result["text"],
        "routing_log": state["routing_log"] + ["research"],
        "total_tokens": state["total_tokens"] + result["tokens"],
        "total_seconds": state["total_seconds"] + result["seconds"],
    }


def write_node(state: TeamState) -> dict:
    result = write(state["research_text"])
    return {
        "summary": result["text"],
        "routing_log": state["routing_log"] + ["write"],
```

```
        "total_tokens": state["total_tokens"] + result["tokens"],
        "total_seconds": state["total_seconds"] + result["seconds"],
    }


def build_graph():
    builder = StateGraph(TeamState)
    builder.add_node("research", research_node)
    builder.add_node("write", write_node)
    builder.add_edge(START, "research")
    builder.add_edge("research", "write")
    builder.add_edge("write", END)
    return builder.compile()


graph = build_graph()


def run_supervisor(topic: str) -> dict:
    initial_state = {
        "task": topic,
        "research_text": "",
        "summary": "",
        "routing_log": [],
        "total_tokens": 0,
        "total_seconds": 0.0,
    }
    final_state = graph.invoke(initial_state)
    return {
        "routing_log": final_state["routing_log"],
        "total_tokens": final_state["total_tokens"],
        "total_seconds": final_state["total_seconds"],
    }


# Compare the sequential and supervisor designs on the same topics.
topics = ["climate change", "inflation", "sleep quality"]
for topic in topics:
    sequential_report = run_sequential(topic)
    supervisor_report = run_supervisor(topic)
    seq_t = sequential_report["total_tokens"]
    seq_s = sequential_report["total_seconds"]
    sup_t = supervisor_report["total_tokens"]
    sup_s = supervisor_report["total_seconds"]
    print(f"\n{topic}")
    print(f"  sequential: {seq_t} tokens, {seq_s:.2f}s")
```

```
    print(f"  supervisor: {sup_t} tokens, {sup_s:.2f}s")
    print(f"  routed: {supervisor_report['routing_log']}")
```

**Expected output** (exact numbers vary by run):

```
climate change
  sequential: 306 tokens, 2.75s
  supervisor: 306 tokens, 2.81s
  routed: ['research', 'write']

inflation
  sequential: 286 tokens, 2.60s
  supervisor: 286 tokens, 2.66s
  routed: ['research', 'write']

sleep quality
  sequential: 325 tokens, 2.90s
  supervisor: 325 tokens, 2.96s
  routed: ['research', 'write']
```